Data_Scientist — Step 1

Learn Python & SQL

CareerCompass • Detailed Learning Notes • Batch 3

1. Learning Objective

Build the coding foundation used to acquire, transform, query and analyze data.

2. Core Concepts — Detailed Breakdown

Python

Learn variables, control flow, functions, modules, exceptions, file handling and environments, then move into NumPy, pandas and notebooks. Write reusable functions instead of one-off notebook cells.

SQL fundamentals

Understand SELECT, filtering, sorting, aggregation, GROUP BY, HAVING, joins, subqueries and common table expressions. SQL is often the first layer of analysis against structured business data.

Data types and schemas

Understand numeric, categorical, text, date/time and identifier fields. A correct query can still produce a wrong result if joins multiply rows or a field has an unexpected meaning.

Reproducibility

Keep queries, notebooks, assumptions and datasets organized. Record data sources and transformations so the analysis can be audited.

3. Deep-Dive Study Notes

Python: what to understand beyond the definition

Python is the implementation layer behind much of the modern data stack. For data work, the goal is not to memorize every language feature; it is to become reliable at transforming inputs into deterministic outputs. That means writing functions with clear inputs and outputs, validating assumptions, handling exceptions intentionally, and keeping analysis code readable enough for another person to audit.

A useful mental model is to separate a data program into four stages: acquire, transform, analyze, and publish. Acquisition may read a CSV, database query or API response. Transformation standardizes types and structure. Analysis computes statistics or models. Publishing produces a table, chart, report or API response. Keeping these stages separate makes debugging much easier because you can inspect the output of each stage.

For numerical work, vectorized operations matter because libraries such as NumPy and pandas operate on arrays efficiently and expose operations at a higher level than manual Python loops. However, vectorization is not magic: you still need to understand shapes, missing values, broadcasting and data types. A common beginner error is to obtain a result that is syntactically valid but semantically wrong because two arrays have compatible shapes but represent different entities.

A professional Python workflow also includes environments and dependency management. Record the Python version and important package versions, keep configuration separate from code, and avoid absolute paths tied to one computer. For repeatable analysis, a fresh environment should be able to install the dependencies and reproduce the result from documented inputs.

Study check: explain this concept without looking at your notes, then apply it to a small dataset or toy example. If you cannot explain the result and its assumptions, return to the underlying concept rather than memorizing the procedure.

SQL fundamentals: what to understand beyond the definition

SQL is a declarative language: you describe the result you want and the database engine determines how to execute the query. The analyst still has to reason about relational structure. Before writing a join, identify the grain of every table—for example, one row per order, one row per order item, or one row per customer. Many incorrect dashboards are caused by joining different grains and unintentionally multiplying rows.

Think of a query as a sequence of logical operations: identify the source rows, filter them, combine tables, group rows when required, calculate aggregates, and finally sort or limit the result. Understanding this logical flow makes clauses such as WHERE, GROUP BY and HAVING easier to reason about. It also helps explain why a condition on an aggregate belongs in HAVING rather than WHERE.

When analyzing business data, always reconcile important totals. If a revenue query returns a number that differs from the source system, do not immediately assume the query is wrong or the source is wrong. Check duplicate keys, null join keys, date filters, currency/unit definitions, refunds, cancelled records and whether revenue is stored at line-item or order level.

Good SQL is readable and testable. Use meaningful aliases, format complex queries, isolate complicated transformations in CTEs when useful, and test each logical component separately. For large datasets, also consider indexes, predicate selectivity and whether aggregation can be pushed to the database rather than transferring unnecessary rows to Python.

Study check: explain this concept without looking at your notes, then apply it to a small dataset or toy example. If you cannot explain the result and its assumptions, return to the underlying concept rather than memorizing the procedure.

4. Recommended Learning Workflow

- Review Python basics.
- Practice pandas DataFrames and missing-data handling.
- Write SQL queries from simple to multi-table.
- Combine SQL extraction with pandas analysis.
- Build one notebook from raw data to conclusion.

5. Worked Practice Plan

Use the following sequence for deliberate practice. First reproduce a small example exactly. Next change one input and predict how the output should change before running the code. Then introduce an imperfect input such as a missing value, duplicate row, unusual category or extreme value. Finally, explain how you validated the result. This four-stage pattern develops debugging and analytical judgment rather than only tool familiarity.

Practice pass 1

Reproduce a minimal example and annotate every important variable, field and assumption.

Practice pass 2

Modify one parameter or subset and predict the result before executing the analysis.

Practice pass 3

Introduce one realistic data-quality problem and decide how it should be handled.

Practice pass 4

Compare the result with an independent calculation, query, chart or known total.

Practice pass 5

Write a short conclusion that separates observation, interpretation and limitation.

6. Hands-on Project / Portfolio Task

Analyze a sales dataset stored in MySQL or SQLite. Write SQL queries for revenue by month/category/customer, export the result to pandas, calculate additional metrics, visualize trends and write five evidence-backed findings.

Project execution checklist

- Write the problem statement and define the unit/grain of the data.
- Create a small data dictionary or schema note before transforming anything.
- Keep raw inputs unchanged and record transformations.
- Create at least one baseline or simple reference result.
- Validate important numbers independently.
- Document one failure or limitation instead of hiding it.
- Publish a reproducible README with tools, setup and expected output.

7. Common Mistakes & Pitfalls

- Using SELECT * everywhere.
- Joining tables without checking cardinality.

- Converting every SQL task into Python when the database can aggregate efficiently.
- Hard-coding assumptions without documentation.
- Confusing identifiers with measures.

8. Completion Checklist

- I can write multi-table SQL queries.
- I can manipulate DataFrames.
- I can identify data types and schema problems.
- I can move data between SQL and Python.
- I can make a reproducible analysis notebook.

9. Interview / Assessment Questions

- What is the difference between WHERE and HAVING?
- Why can a join duplicate rows?
- When should aggregation happen in SQL?
- What is a DataFrame?
- How do you handle missing values?

10. Mini Revision Sheet

Before considering this step complete, be able to define the key terms, explain why the technique exists, identify when it should not be used, perform a small example without copying a tutorial, validate the output, and explain one real-world use case. A strong learner can answer 'why?' as well as 'how?'.

11. Recommended References

Python Tutorial: https://docs.python.org/3/tutorial/

pandas User Guide: https://pandas.pydata.org/docs/user_guide/

Data Science Tutorial: https://www.youtube.com/watch?v=ua-CiDNNj30

Data Analyst Bootcamp: https://www.youtube.com/watch?v=PSNXoAs2FtQ

Research note: The learning structure was expanded using current official documentation and free learning resources. Reference links are included so learners can verify current APIs, workflows and certification/training details.